

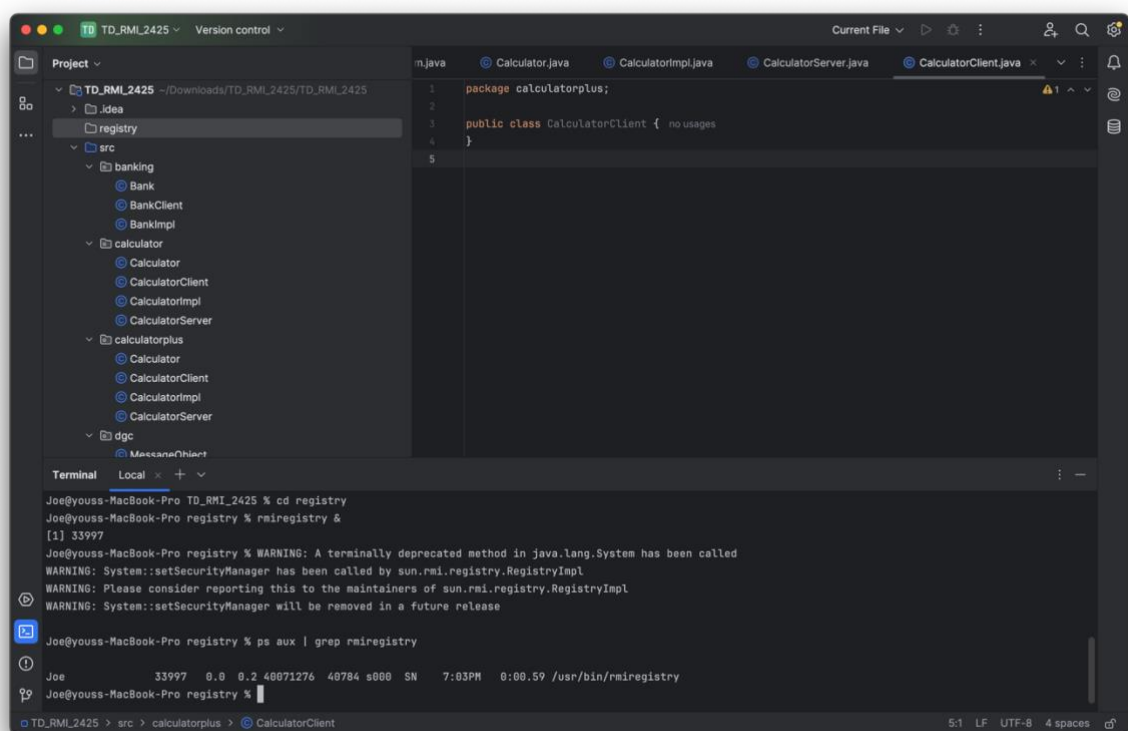
Youssef SAADE
220958

github link :

https://github.com/Yousseffsaade/applications_distribuees.git

branch : Implémentation-d'un-service-distribué-Java-RMI

Implémentation d'un service distribué Java RMI

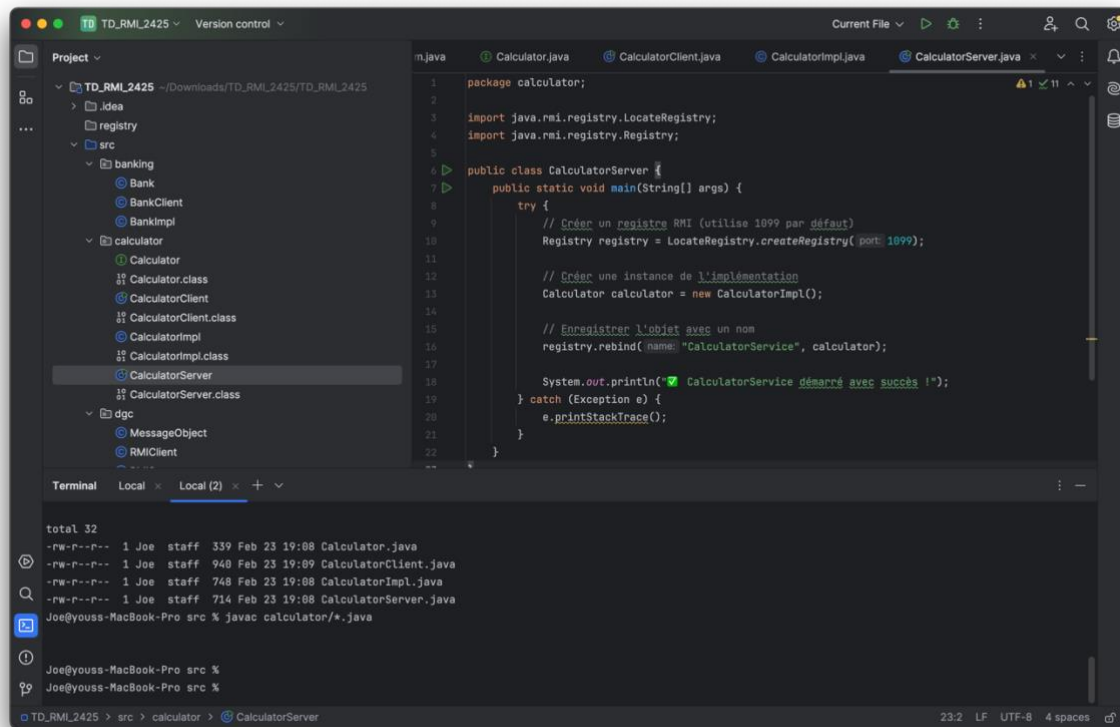


```
package calculatorplus;

public class CalculatorClient { no usages
}

Terminal
Local x + v
Joe@yousse-MacBook-Pro TD_RMI_2425 % cd registry
Joe@yousse-MacBook-Pro registry % rmiregistry &
[1] 33997
Joe@yousse-MacBook-Pro registry % WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by sun.rmi.registry.RegistryImpl
WARNING: Please consider reporting this to the maintainers of sun.rmi.registry.RegistryImpl
WARNING: System::setSecurityManager will be removed in a future release
Joe@yousse-MacBook-Pro registry % ps aux | grep rmiregistry
Joe      33997  0.0  0.2 40071276 40784 s000  SN   7:03PM  0:00.59 /usr/bin/rmiregistry
Joe@yousse-MacBook-Pro registry %
```

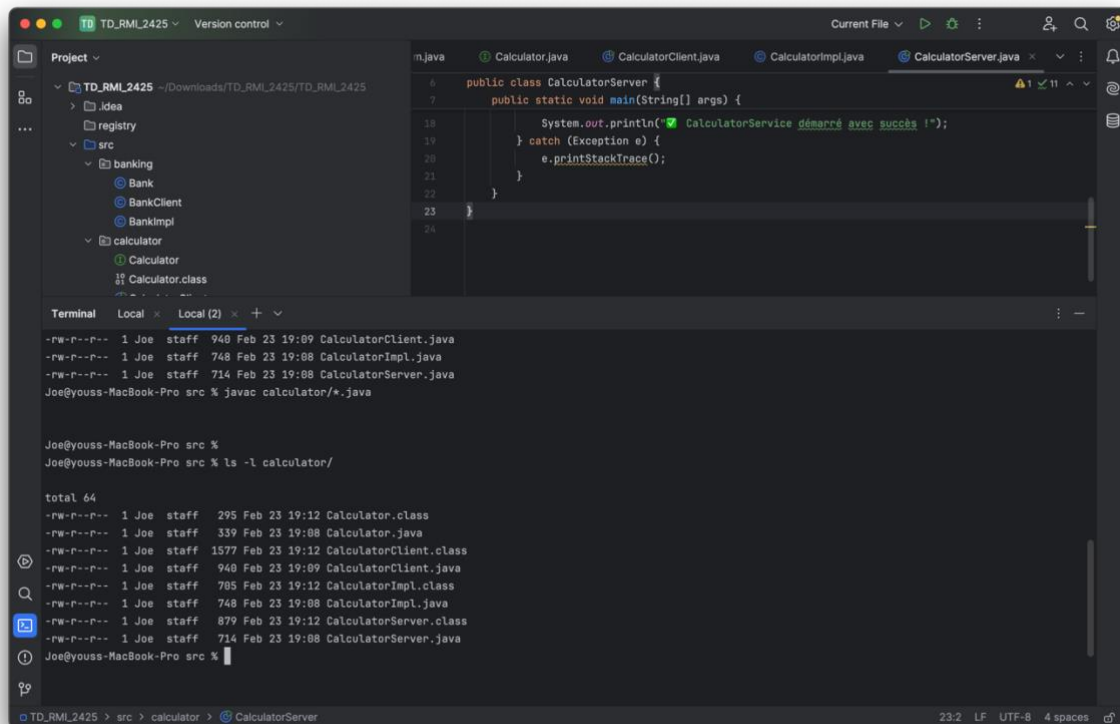
Calculator



The screenshot shows an IDE with the following components:

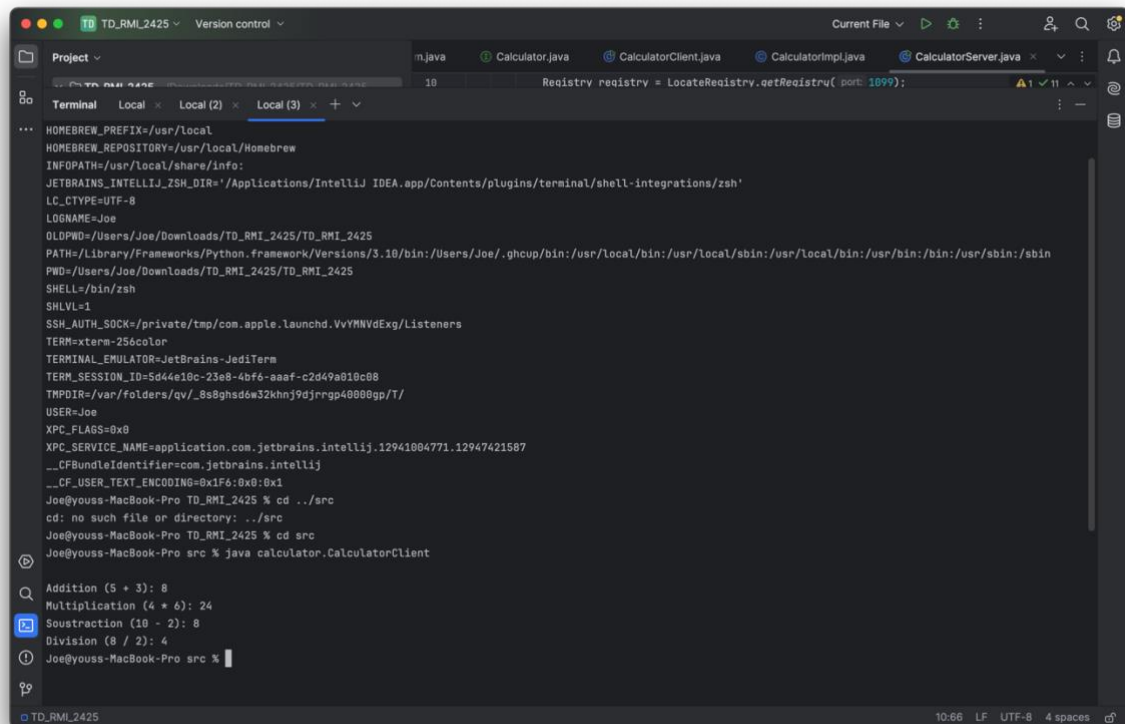
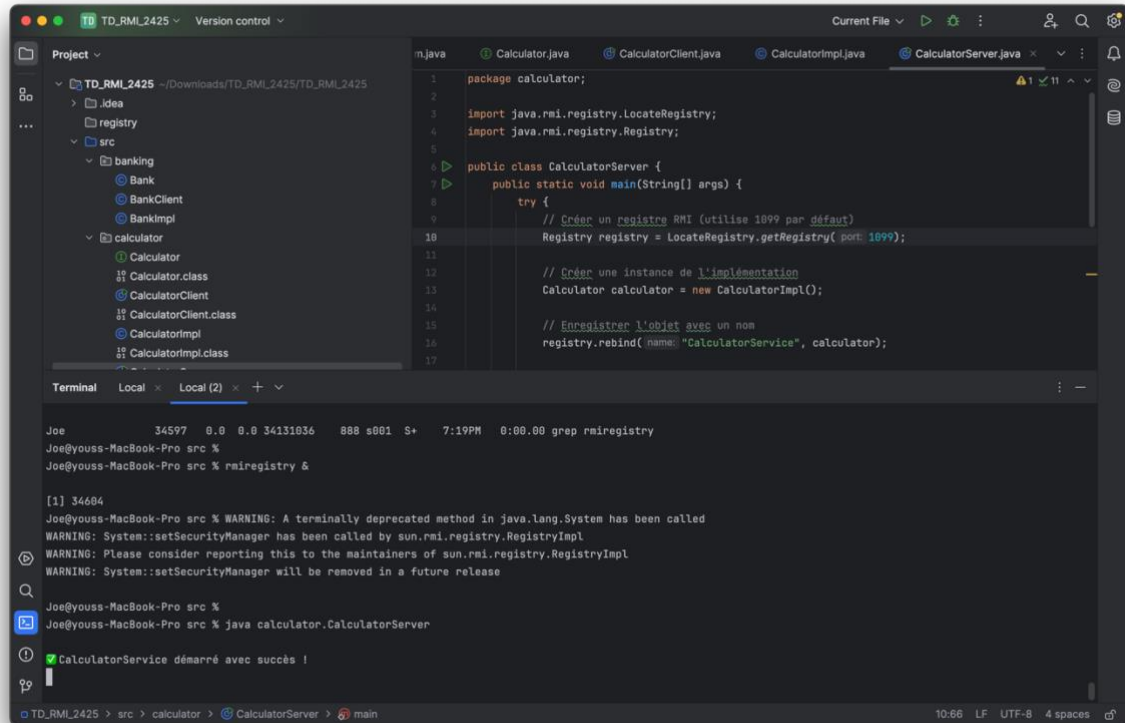
- Project View:** A tree structure on the left showing the project layout. The 'calculator' package is expanded, showing files: Calculator.class, CalculatorClient.class, CalculatorImpl.class, and CalculatorServer.class.
- Editor:** The 'CalculatorServer.java' file is open. It contains the following code:

```
1 package calculator;
2
3 import java.rmi.registry.LocateRegistry;
4 import java.rmi.registry.Registry;
5
6 public class CalculatorServer {
7     public static void main(String[] args) {
8         try {
9             // Créer un registre RMI (utilise 1099 par défaut)
10            Registry registry = LocateRegistry.createRegistry(1099);
11
12            // Créer une instance de l'implémentation
13            Calculator calculator = new CalculatorImpl();
14
15            // Enregistrer l'objet avec un nom
16            registry.rebind(name: "CalculatorService", calculator);
17
18            System.out.println("✓ CalculatorService démarré avec succès !");
19        } catch (Exception e) {
20            e.printStackTrace();
21        }
22    }
23 }
```
- Terminal:** The terminal shows the output of the compilation command 'javac calculator/*.java'. It lists the files compiled: CalculatorClient.java, CalculatorImpl.java, and CalculatorServer.java. The output ends with 'total 32'.



The screenshot shows the same IDE as the first one, but with the following changes:

- Editor:** The 'CalculatorServer.java' file is still open, but the code is now empty, showing only line numbers 6 through 24.
- Terminal:** The terminal shows the output of the compilation command 'javac calculator/*.java'. It lists the files compiled: CalculatorClient.java, CalculatorImpl.java, and CalculatorServer.java. The output ends with 'total 64'.



Dans ce projet, nous avons mis en place un service RMI (Remote Method Invocation) permettant d'effectuer des opérations arithmétiques à distance.

1 Démarrage du registre RMI (rmiregistry) pour permettre la communication entre le serveur et le client.

2 Compilation du code source (Calculator.java, CalculatorImpl.java, CalculatorServer.java, CalculatorClient.java).

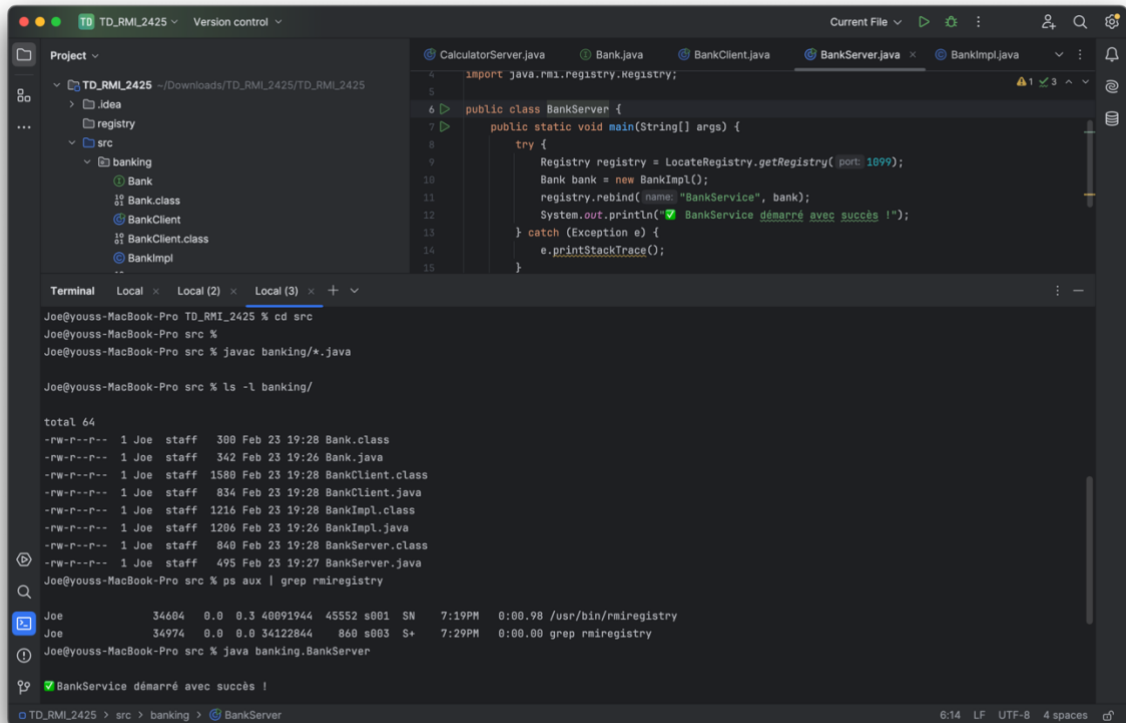
3 Exécution du serveur (CalculatorServer), qui enregistre le service RMI.

4 Exécution du client (CalculatorClient), qui effectue des appels distants au serveur pour réaliser des calculs.

5 Vérification des résultats : les calculs ont été effectués avec succès et affichés correctement par le client.

Le service fonctionne sans erreur après la bonne organisation des fichiers et la configuration correcte du registre RMI.

Service Bancaire (Banking Service RMI)



```
import java.rmi.registry.Registry;

public class BankServer {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry(port: 1099);
            Bank bank = new BankImpl();
            registry.rebind(name: "BankService", bank);
            System.out.println("BankService démarré avec succès !");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
Joe@youss-MacBook-Pro TD_RMI_2425 % cd src
Joe@youss-MacBook-Pro src % javac banking/*.java

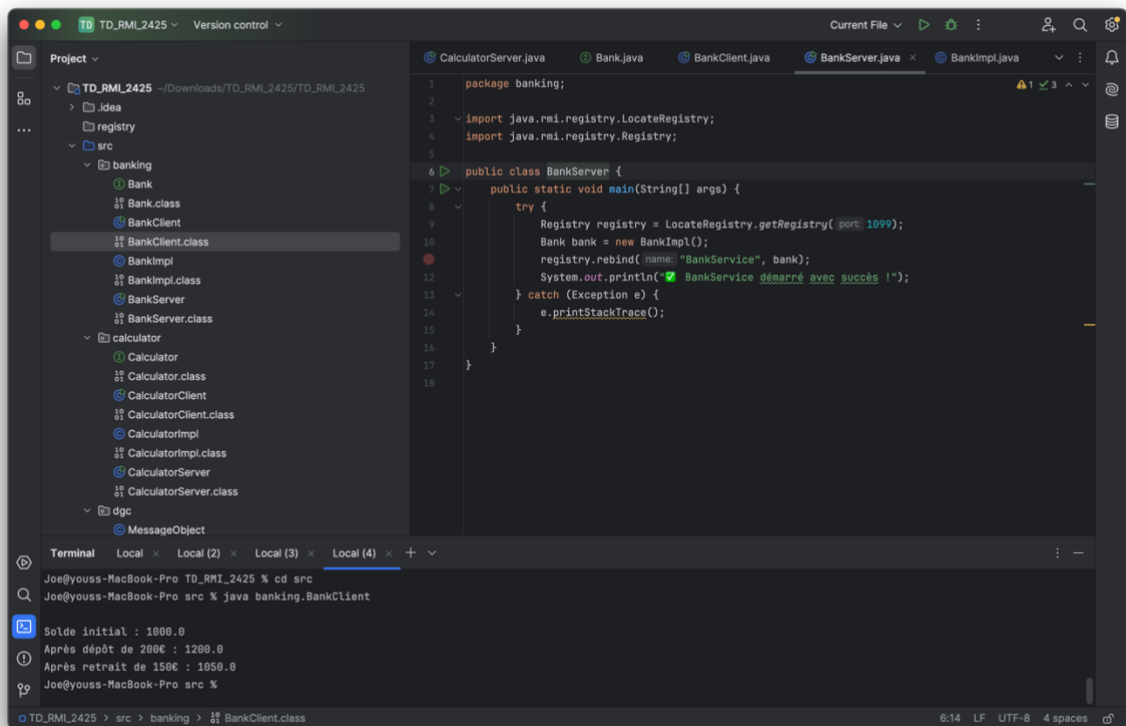
Joe@youss-MacBook-Pro src % ls -l banking/

total 64
-rw-r--r--  1 Joe  staff   308 Feb 23 19:28 Bank.class
-rw-r--r--  1 Joe  staff   342 Feb 23 19:26 Bank.java
-rw-r--r--  1 Joe  staff  1580 Feb 23 19:28 BankClient.class
-rw-r--r--  1 Joe  staff   834 Feb 23 19:28 BankClient.java
-rw-r--r--  1 Joe  staff  1216 Feb 23 19:28 BankImpl.class
-rw-r--r--  1 Joe  staff  1206 Feb 23 19:26 BankImpl.java
-rw-r--r--  1 Joe  staff   840 Feb 23 19:28 BankServer.class
-rw-r--r--  1 Joe  staff   495 Feb 23 19:27 BankServer.java
Joe@youss-MacBook-Pro src % ps aux | grep rmiregistry

Joe          34604  0.0  0.3 40091944 45552 s001  SN   7:19PM  0:00.98  /usr/bin/rmiregistry
Joe          34974  0.0  0.0 34122844   860 s003  S+   7:29PM  0:00.00  grep rmiregistry

Joe@youss-MacBook-Pro src % java banking.BankServer

BankService démarré avec succès !
```



```
package banking;

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class BankClient {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry(port: 1099);
            Bank bank = new BankImpl();
            registry.rebind(name: "BankService", bank);
            System.out.println("BankService démarré avec succès !");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
Joe@youss-MacBook-Pro TD_RMI_2425 % cd src
Joe@youss-MacBook-Pro src % java banking.BankClient

Solde initial : 1000.0
Après dépôt de 200€ : 1200.0
Après retrait de 150€ : 1050.0
Joe@youss-MacBook-Pro src %
```

Dans cette partie, nous avons mis en place un service bancaire distribué en utilisant Java RMI.

1 Compilation des fichiers sources (Bank.java, BankImpl.java, BankServer.java, BankClient.java).

2 Démarrage du serveur bancaire (BankServer), qui enregistre le service BankService.

3 Vérification de la mise en place du registre RMI (rmiregistry).

4 Exécution du client bancaire (BankClient), qui effectue des opérations sur le compte (dépôt, retrait).

5 Résultats affichés correctement :

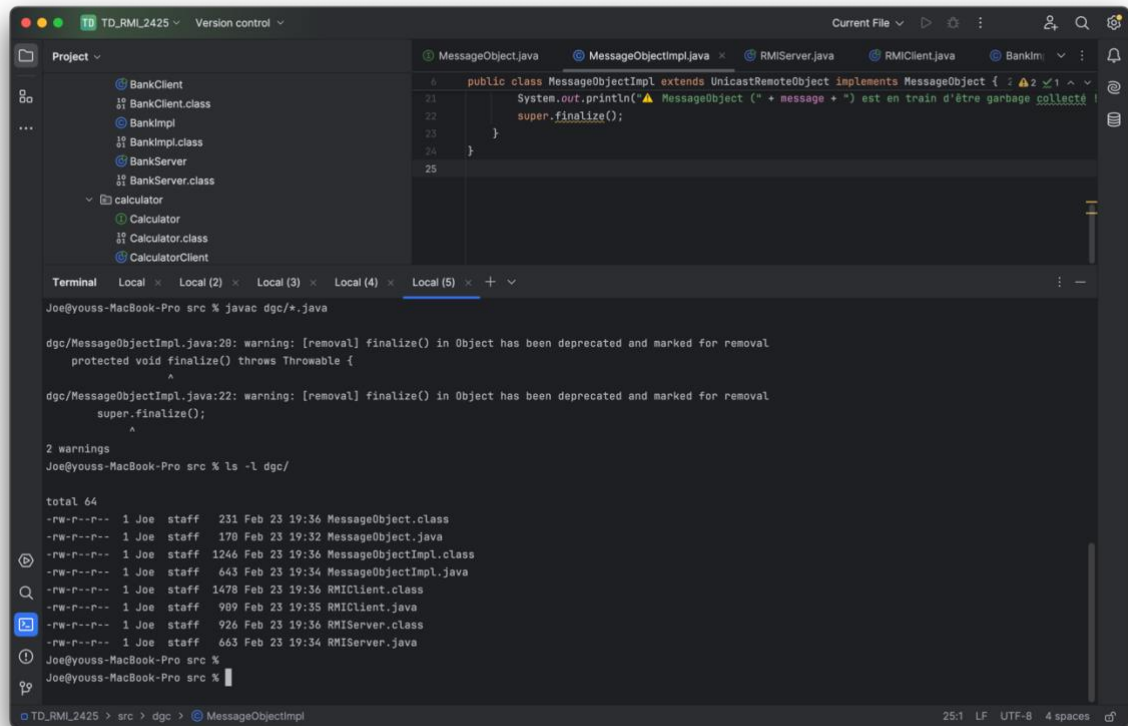
Solde initial : 1000.0

Après dépôt de 200€ : 1200.0

Après retrait de 150€ : 1050.0

Le service bancaire fonctionne correctement et permet d'interagir avec un compte bancaire distant via RMI.

Distributed Garbage Collection (DGC) avec RMI



The screenshot shows an IDE with a project named 'TD_RMI_2425'. The project structure includes a 'calculator' package with 'Calculator' and 'CalculatorClient' classes, and a 'dgc' package with 'MessageObject' and 'MessageObjectImpl' classes. The 'MessageObjectImpl.java' file is open, showing a class that extends 'UnicastRemoteObject' and implements 'MessageObject'. It has a 'finalize()' method that prints a message and calls 'super.finalize()'. The terminal shows the compilation command 'javac dgc/*.java' and the output, which includes two warnings about the deprecated 'finalize()' method. The file list shows the compiled classes and their sizes.

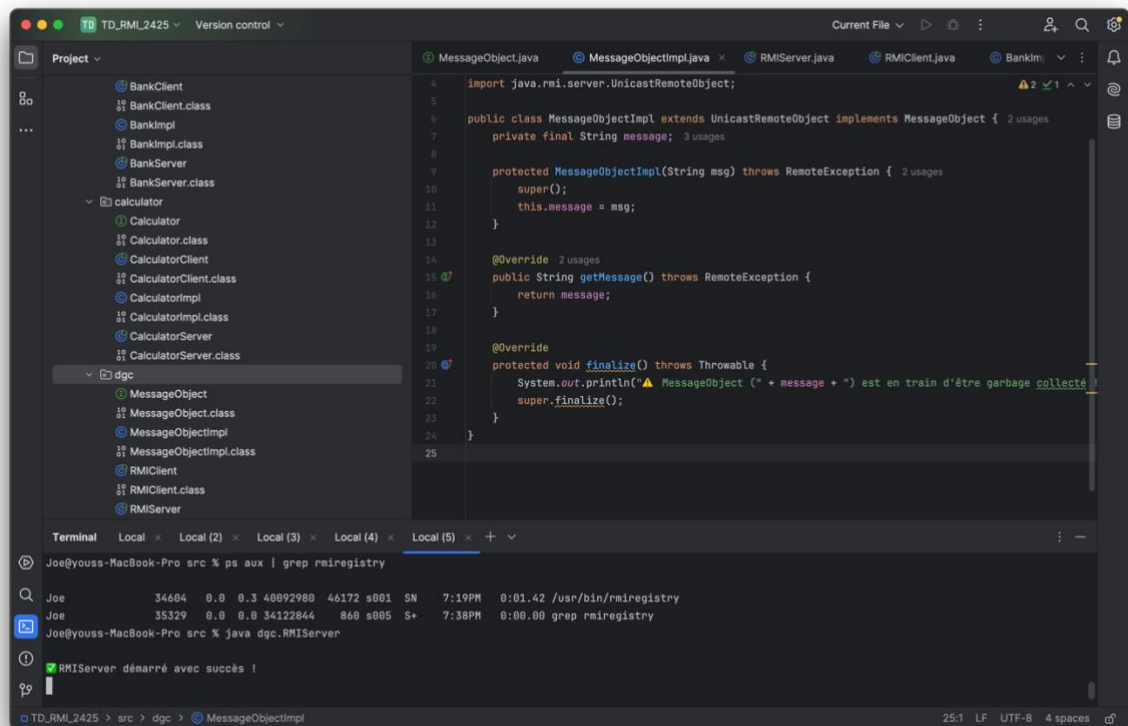
```
Project
├── BankClient
│   ├── BankClient.class
│   ├── BankImpl
│   │   ├── BankImpl.class
│   │   ├── BankServer
│   │   │   ├── BankServer.class
│   │   └── calculator
│   │       ├── Calculator
│   │       │   ├── Calculator.class
│   │       │   └── CalculatorClient
│   │       └── dgc
│           ├── MessageObject
│           │   ├── MessageObject.class
│           │   ├── MessageObjectImpl
│           │   │   ├── MessageObjectImpl.class
│           │   ├── RMIClient
│           │   │   ├── RMIClient.class
│           │   └── RMIServer
│           │   │   ├── RMIServer.class
│           └── RMIServer
```

```
MessageObjectImpl.java
1  public class MessageObjectImpl extends UnicastRemoteObject implements MessageObject {
2      System.out.println("▲ MessageObject (" + message + ") est en train d'être garbage collecté !");
3      super.finalize();
4  }
5  }
```

```
Terminal
Joe@youss-MacBook-Pro src % javac dgc/*.java

dgc/MessageObjectImpl.java:28: warning: [removal] finalize() in Object has been deprecated and marked for removal
protected void finalize() throws Throwable {
      ^
dgc/MessageObjectImpl.java:22: warning: [removal] finalize() in Object has been deprecated and marked for removal
super.finalize();
      ^
2 warnings
Joe@youss-MacBook-Pro src % ls -l dgc/

total 64
-rw-r--r-- 1 Joe staff 231 Feb 23 19:36 MessageObject.class
-rw-r--r-- 1 Joe staff 178 Feb 23 19:32 MessageObject.java
-rw-r--r-- 1 Joe staff 1246 Feb 23 19:36 MessageObjectImpl.class
-rw-r--r-- 1 Joe staff 643 Feb 23 19:34 MessageObjectImpl.java
-rw-r--r-- 1 Joe staff 1478 Feb 23 19:36 RMIClient.class
-rw-r--r-- 1 Joe staff 909 Feb 23 19:35 RMIClient.java
-rw-r--r-- 1 Joe staff 926 Feb 23 19:36 RMIServer.class
-rw-r--r-- 1 Joe staff 663 Feb 23 19:34 RMIServer.java
Joe@youss-MacBook-Pro src %
Joe@youss-MacBook-Pro src %
```

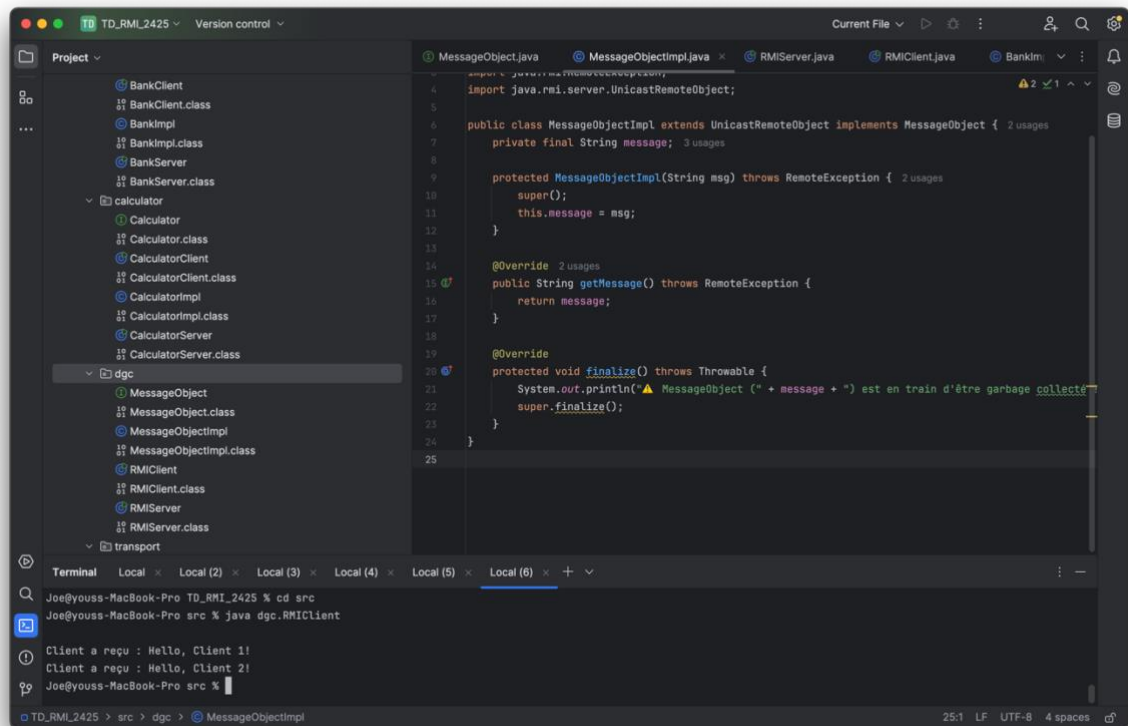


The screenshot shows the same IDE with the 'dgc' package selected in the project structure. The 'MessageObjectImpl.java' file is open, showing the same code as before. The terminal shows the command 'ps aux | grep rmiregistry' and the output, which shows the 'rmiregistry' process running. The command 'java dgc.RMIServer' is also shown, and the output indicates that the 'RMIServer' has started successfully.

```
Project
├── BankClient
│   ├── BankClient.class
│   ├── BankImpl
│   │   ├── BankImpl.class
│   │   ├── BankServer
│   │   │   ├── BankServer.class
│   │   └── calculator
│   │       ├── Calculator
│   │       │   ├── Calculator.class
│   │       │   └── CalculatorClient
│   │       └── dgc
│           ├── MessageObject
│           │   ├── MessageObject.class
│           │   ├── MessageObjectImpl
│           │   │   ├── MessageObjectImpl.class
│           │   ├── RMIClient
│           │   │   ├── RMIClient.class
│           │   └── RMIServer
│           │   │   ├── RMIServer.class
│           └── RMIServer
```

```
MessageObjectImpl.java
1  import java.rmi.server.UnicastRemoteObject;
2
3  public class MessageObjectImpl extends UnicastRemoteObject implements MessageObject {
4      private final String message;
5
6      protected MessageObjectImpl(String msg) throws RemoteException {
7          super();
8          this.message = msg;
9      }
10
11      @Override
12      public String getMessage() throws RemoteException {
13          return message;
14      }
15
16      @Override
17      protected void finalize() throws Throwable {
18          System.out.println("▲ MessageObject (" + message + ") est en train d'être garbage collecté !");
19          super.finalize();
20      }
21  }
22  }
```

```
Terminal
Joe@youss-MacBook-Pro src % ps aux | grep rmiregistry
Joe      34684  0.0  0.3 48892988 46172 s001  SN   7:19PM  0:01.42 /usr/bin/rmiregistry
Joe      35329  0.0  0.0 34122844   868 s005  S+   7:38PM  0:00.00 grep rmiregistry
Joe@youss-MacBook-Pro src % java dgc.RMIServer
RMIServer démarré avec succès !
```



Dans cette partie, nous avons mis en place un service RMI illustrant le Distributed Garbage Collection (DGC), qui permet de gérer la suppression automatique des objets distants lorsqu'ils ne sont plus référencés.

1 Compilation des fichiers sources (MessageObject.java, MessageObjectImpl.java, RMIServer.java, RMIClient.java).

2 Démarrage du serveur (RMIServer), qui enregistre un objet distant MessageObjectImpl.

3 Vérification du registre RMI (rmiregistry).

4 Exécution du client (RMIClient), qui interagit avec l'objet distant et affiche les messages reçus.

5 Test du Garbage Collector : affichage d'un message lorsque l'objet distant est supprimé.

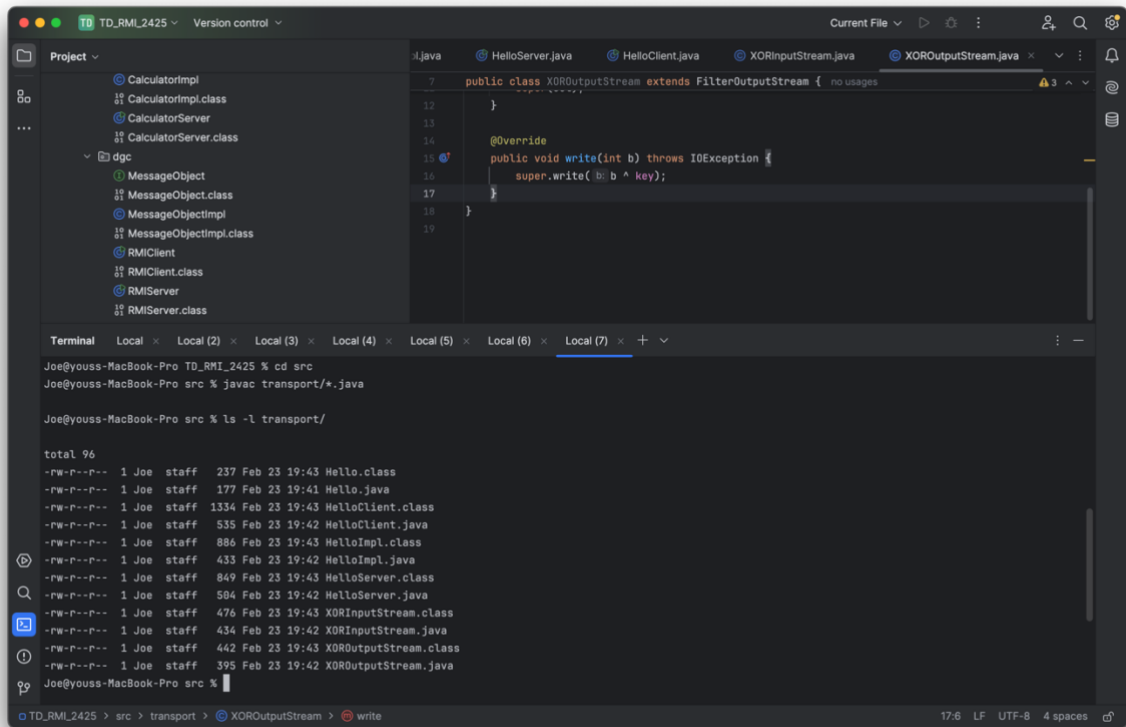
Résultats observés :

Le serveur RMI démarre avec succès.

Le client reçoit les messages distants correctement.

Le message de garbage collection (MessageObject est en train d'être garbage collecté) s'affiche lorsque l'objet est supprimé.

Transport sécurisé avec XOR

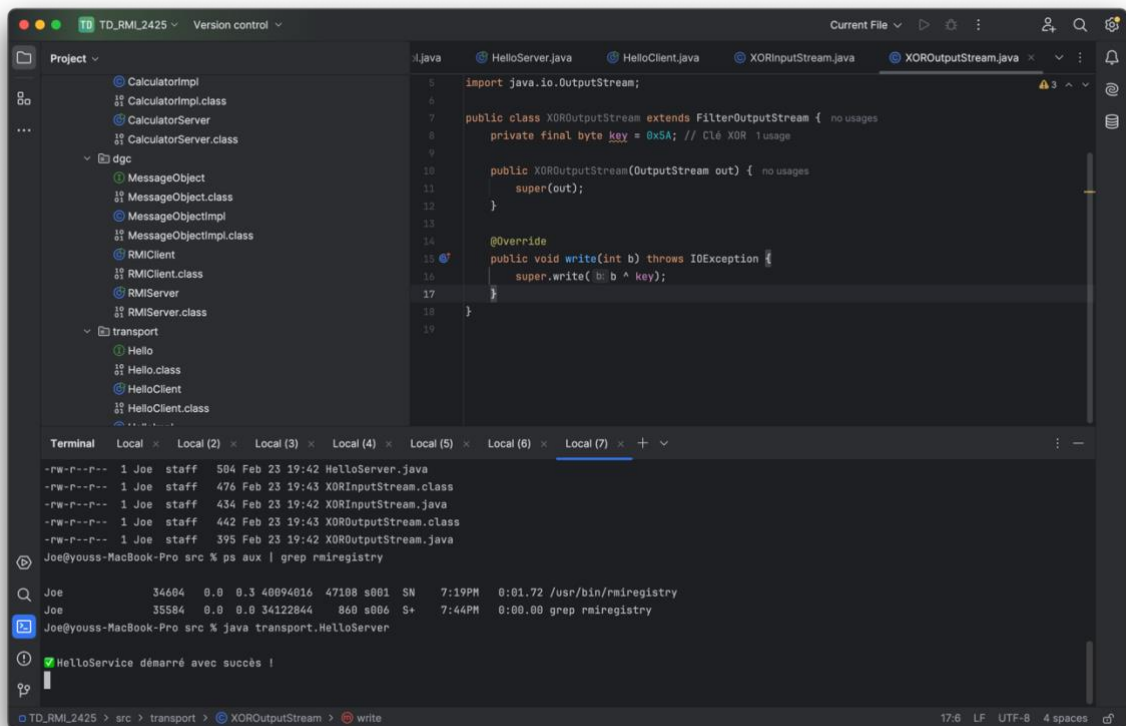


The screenshot shows an IDE with the following components:

- Project Explorer:** Shows a project structure with packages `Calculator` and `dgc`. The `dgc` package contains `MessageObject`, `MessageObjectImpl`, `RMIClient`, `RMIServer`, and `RMIServer.class`.
- Code Editor:** Displays the `XOROutputStream.java` file. The code is as follows:

```
7 public class XOROutputStream extends FilterOutputStream {  
12 }  
13  
14 @Override  
15 public void write(int b) throws IOException {  
16     super.write(b & b ^ key);  
17 }  
18  
19 }
```
- Terminal:** Shows the following commands and output:

```
Joe@youss-MacBook-Pro TD_RMI_2425 % cd src  
Joe@youss-MacBook-Pro src % javac transport/*.java  
Joe@youss-MacBook-Pro src % ls -l transport/  
  
total 96  
-rw-r--r--  1 Joe  staff   237 Feb 23 19:43 Hello.class  
-rw-r--r--  1 Joe  staff   177 Feb 23 19:41 Hello.java  
-rw-r--r--  1 Joe  staff  1334 Feb 23 19:43 HelloClient.class  
-rw-r--r--  1 Joe  staff   535 Feb 23 19:42 HelloClient.java  
-rw-r--r--  1 Joe  staff   886 Feb 23 19:43 HelloImpl.class  
-rw-r--r--  1 Joe  staff   433 Feb 23 19:42 HelloImpl.java  
-rw-r--r--  1 Joe  staff   849 Feb 23 19:43 HelloServer.class  
-rw-r--r--  1 Joe  staff   504 Feb 23 19:42 HelloServer.java  
-rw-r--r--  1 Joe  staff   476 Feb 23 19:43 XORInputStream.class  
-rw-r--r--  1 Joe  staff   434 Feb 23 19:42 XORInputStream.java  
-rw-r--r--  1 Joe  staff   442 Feb 23 19:43 XOROutputStream.class  
-rw-r--r--  1 Joe  staff   395 Feb 23 19:42 XOROutputStream.java  
Joe@youss-MacBook-Pro src %
```

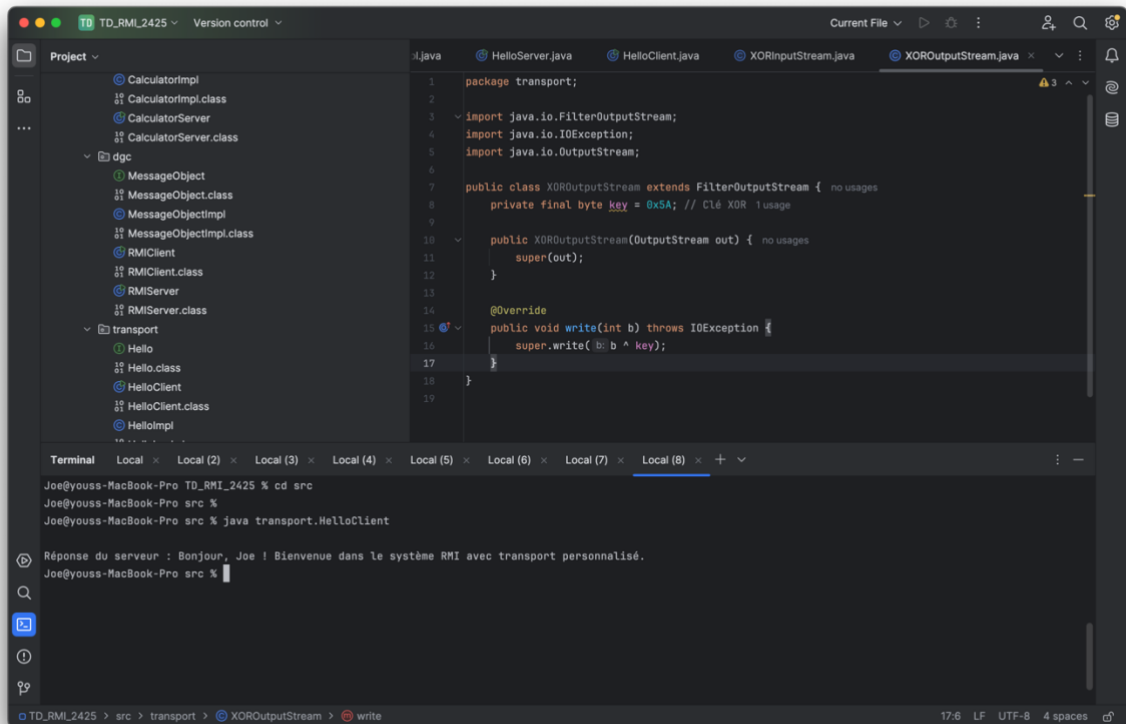


The screenshot shows the IDE with the following components:

- Project Explorer:** The `transport` package is now visible, containing `Hello`, `Hello.class`, `HelloClient`, and `HelloClient.class`.
- Code Editor:** Displays the updated `XOROutputStream.java` file. The code is as follows:

```
5 import java.io.OutputStream;  
6  
7 public class XOROutputStream extends FilterOutputStream {  
8     private final byte key = 0x5A; // Clé XOR 1 usage  
9  
10    public XOROutputStream(OutputStream out) {  
11        super(out);  
12    }  
13  
14    @Override  
15    public void write(int b) throws IOException {  
16        super.write(b & b ^ key);  
17    }  
18  
19 }
```
- Terminal:** Shows the following commands and output:

```
-rw-r--r--  1 Joe  staff   504 Feb 23 19:42 HelloServer.java  
-rw-r--r--  1 Joe  staff   476 Feb 23 19:43 XORInputStream.class  
-rw-r--r--  1 Joe  staff   434 Feb 23 19:42 XORInputStream.java  
-rw-r--r--  1 Joe  staff   442 Feb 23 19:43 XOROutputStream.class  
-rw-r--r--  1 Joe  staff   395 Feb 23 19:42 XOROutputStream.java  
Joe@youss-MacBook-Pro src % ps aux | grep rmiregistry  
  
Joe          34684  0.0  0.3 48094816  47108 s001  SN   7:19PM  0:01.72 /usr/bin/rmiregistry  
Joe          35586  0.0  0.0 34122844   860 s006  S+   7:44PM  0:00.00 grep rmiregistry  
Joe@youss-MacBook-Pro src % java transport.HelloServer  
  
✓ HelloService démarré avec succès !  
Joe@youss-MacBook-Pro src %
```



Cette partie du projet met en place un système de transport sécurisé en utilisant un chiffrement XOR pour protéger les communications RMI.

1 Compilation et organisation des fichiers

Tous les fichiers du package transport (Hello.java, HelloClient.java, HelloServer.java, HelloImpl.java, XORInputStream.java, XOROutputStream.java) ont été compilés avec succès.

Vérification des fichiers .class générés.

2 Mise en place du serveur HelloServer

Démarrage du registre RMI.

Exécution du serveur HelloServer qui expose un service distant permettant de renvoyer un message personnalisé.

Confirmation que le HelloService a démarré avec succès.

3 Lancement du client HelloClient

Connexion au serveur via RMI avec un transport personnalisé basé sur XOR.

Réception de la réponse sécurisée du serveur :

Réponse du serveur : Bonjour, Joe ! Bienvenue dans le système RMI avec transport personnalisé.

Partie Bonus - CalculatorPlus

The screenshot shows an IDE with a project named 'TD_RMI_2425'. The 'Project' view on the left shows the directory structure: 'TD_RMI_2425' > 'client' > 'CalculatorPlus'. The 'Terminal' tab is active, showing the following commands and output:

```
Joe@youss-MacBook-Pro server % ps aux | grep rmiregistry
Joe      36861  0.0  0.0 3412844   860 s011  S+   8:16PM  0:00.00 grep rmiregistry
Joe      36659  0.0  0.3 40861544 44392 s011  SN   8:11PM  0:00.69 /usr/bin/rmiregistry
Joe@youss-MacBook-Pro server % kill -9 36659

Joe@youss-MacBook-Pro server %
[1] + killed    rmiregistry
Joe@youss-MacBook-Pro server %
Joe@youss-MacBook-Pro server % cd /Users/Joe/Downloads/TD_RMI_2425/CalculatorPlus/server
rmiregistry &

[1] 36882
Joe@youss-MacBook-Pro server %
Joe@youss-MacBook-Pro server % WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by sun.rmi.registry.RegistryImpl
WARNING: Please consider reporting this to the maintainers of sun.rmi.registry.RegistryImpl
WARNING: System::setSecurityManager will be removed in a future release

Joe@youss-MacBook-Pro server %
Joe@youss-MacBook-Pro server % javac calculator/*.java
java -cp . calculator.CalculatorServer

CalculatorService démarré avec succès !
```

The IDE also shows the 'RMIClient.java' file in the editor, which contains the following code:

```
public class RMIClient {
    // ...
    Thread.sleep( milliseconds: 5000 ); // Attend pour observer le GC
    } catch (Exception e) {
    }
    e.printStackTrace();
}
```

The screenshot shows the same IDE with the 'client' directory selected in the 'Project' view. The 'Terminal' tab is active, showing the following commands and output:

```
zsh: no matches found: calculator/*.java
Joe@youss-MacBook-Pro client %
Joe@youss-MacBook-Pro client % ls -R

Calculator.java      CalculatorClient.java
Joe@youss-MacBook-Pro client %
Joe@youss-MacBook-Pro client % mkdir -p calculator

Joe@youss-MacBook-Pro client % mv Calculator*.java calculator/

Joe@youss-MacBook-Pro client % ls -R

calculator

./calculator:
Calculator.java      CalculatorClient.java
Joe@youss-MacBook-Pro client % javac calculator/*.java

Joe@youss-MacBook-Pro client % ls calculator/

Calculator.class      Calculator.java      CalculatorClient.class  CalculatorClient.java
Joe@youss-MacBook-Pro client % java -cp . calculator.CalculatorClient

Addition (5 + 3): 8
Multiplication (4 * 6): 24
Soustraction (10 - 2): 8
Division (8 / 2): 4
Joe@youss-MacBook-Pro client %
```

The IDE also shows the 'RMIClient.java' file in the editor, which contains the following code:

```
public class RMIClient {
    // ...
    Thread.sleep( milliseconds: 5000 ); // Attend pour observer le GC
    } catch (Exception e) {
    }
    e.printStackTrace();
}
```

1 Redémarrage du registre RMI et lancement du serveur

(Capture d'écran 1 - CalculatorService démarré avec succès)

Nous avons d'abord vérifié si un registre RMI était déjà actif avec :

```
ps aux | grep rmiregistry
```

Après avoir tué le processus actif (port 1099 bloqué), nous avons redémarré rmiregistry dans le bon dossier :

```
rmiregistry &
```

Ensuite, nous avons compilé les fichiers Java :

```
javac calculator/*.java
```

Enfin, nous avons lancé le serveur avec :

```
java -cp . calculator.CalculatorServer
```



Message de confirmation affiché :

CalculatorService démarré avec succès !

Cela indique que le serveur est prêt à recevoir des requêtes.

2 Lancement du client et exécution des calculs

(Capture d'écran 2 - Lancement du client et exécution des opérations)

- Après avoir organisé les fichiers sources (déplacement des fichiers du client dans un sous-dossier **calculator/**), nous avons recompilé le client avec :

```
javac calculator/*.java
```

- Ensuite, nous avons exécuté le client avec :

```
java -cp . calculator.CalculatorClient
```

- Les résultats des calculs ont été affichés :

Addition (5 + 3) : 8

Multiplication (4 * 6) : 24

Soustraction (10 - 2) : 8

Division (8 / 2) : 4

 Cela prouve que le client peut communiquer avec le serveur via RMI et exécuter les opérations correctement.